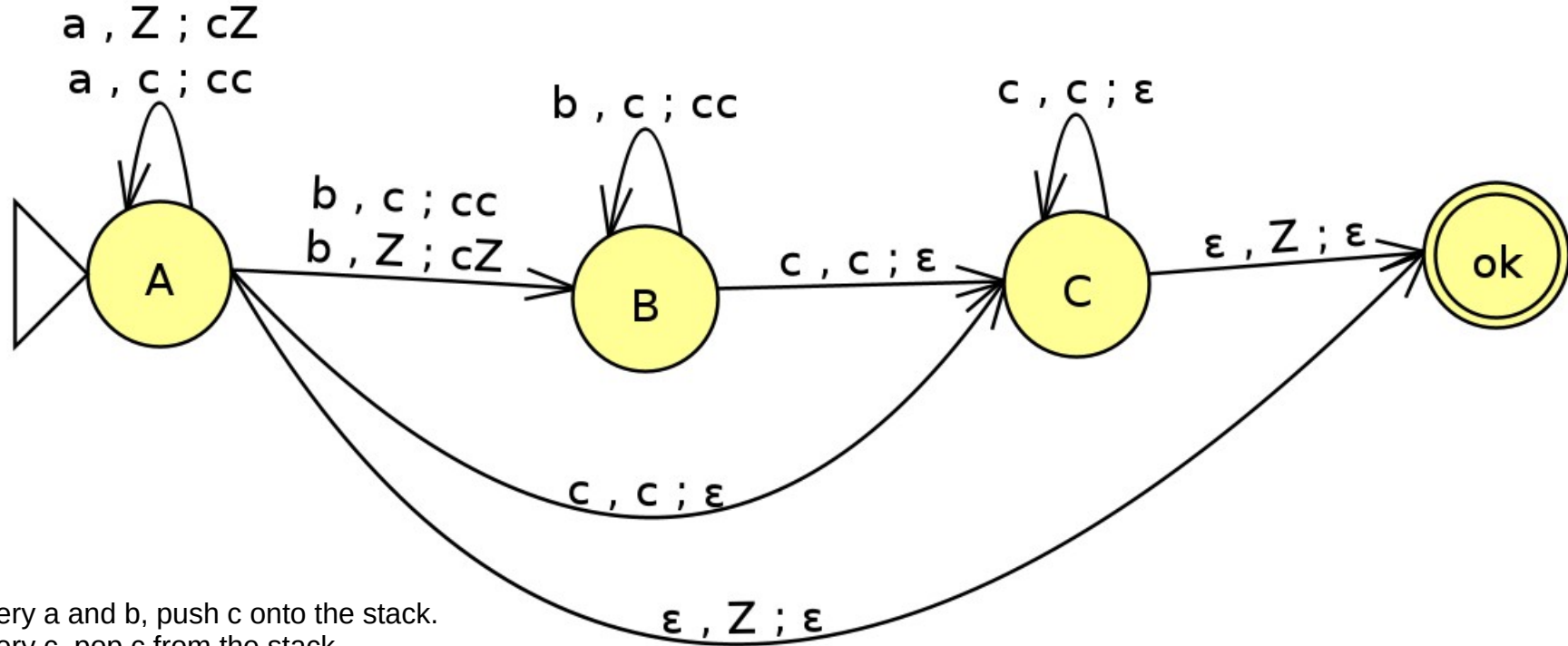


X4 T1. Let's define the words accepted by a language such that the language includes words $\{a^k b^l c^{k+l} \mid k, l \in \mathbb{N}\}$. For example aabbccccc.
 Define a pushdown automaton that accepts the language.



For every a and b, push c onto the stack.
 For every c, pop c from the stack.

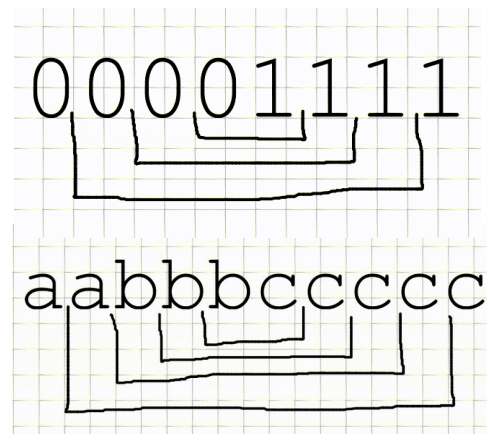
nop/peek is done by taking the topmost from the stack and then putting the same symbol back onto the stack
 (stack doesn't change)

X4 T2. Define a grammar for the language $\{a^k b^l c^{k+l} \mid k, l \in \mathbb{N}\}$

$S \rightarrow A \mid \varepsilon$
 $A \rightarrow aAc \mid \varepsilon \mid B$
 $B \rightarrow bBc \mid \varepsilon$

$S \rightarrow aAC \mid bBC \mid \varepsilon$
 $A \rightarrow aAC \mid bBC \mid \varepsilon$
 $B \rightarrow bBC \mid \varepsilon$
 $C \rightarrow c$

$S \rightarrow aA \mid bB \mid \varepsilon$
 $A \rightarrow aAC \mid bBC \mid c$
 $B \rightarrow bBC \mid c$
 $C \rightarrow c$



a c
 a c
 b c
 ^
 bc

The formation of the string can be thought of as new “ac” pairs being added into the middle of the string, followed later by new “bc” pairs being added into the middle of the string.

X4 T2. Define a grammar for the language $\{a^k b^l c^{k+l} \mid k, l \in \mathbb{N}\}$

$S \rightarrow A \mid \varepsilon$

$A \rightarrow aAc \mid \varepsilon \mid B$

$B \rightarrow bBc \mid \varepsilon$

$S \rightarrow aAC \mid bBC \mid \varepsilon$

$A \rightarrow aAC \mid bBC \mid \varepsilon$

$B \rightarrow bBC \mid \varepsilon$

$C \rightarrow c$

$S \rightarrow aA \mid bB \mid \varepsilon$

$A \rightarrow aAC \mid bBC \mid c$

$B \rightarrow bBC \mid c$

$C \rightarrow c$

Grammar to (non-deterministic) PDA

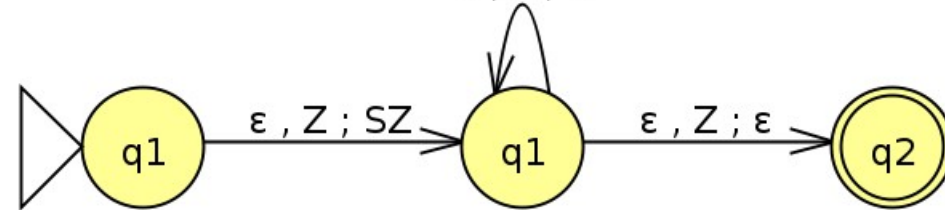
for each production $X \rightarrow \alpha$

- pop X from stack
- push α to stack

for each terminal symbol t

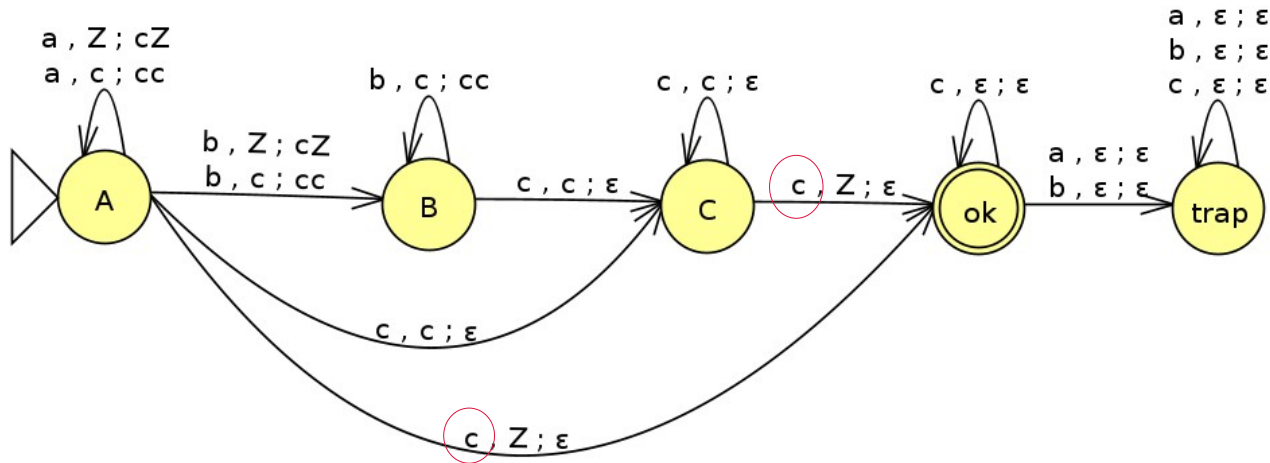
- pop t from stack
- read t from input

$\varepsilon, S; aA$
 $\varepsilon, S; bB$
 $\varepsilon, S; \varepsilon$
 $\varepsilon, A; aAC$
 $\varepsilon, A; bBC$
 $\varepsilon, A; c$
 $\varepsilon, B; bBC$
 $\varepsilon, B; c$
 $\varepsilon, C; c$
 $a, a; \varepsilon$
 $b, b; \varepsilon$
 $c, c; \varepsilon$



X4 T3 Let's define the words accepted by a language such that the language includes words $\{a^k b^l c^m \mid k+l \leq m, k, l, m \in \mathbb{N}\}$.
For example abccccc.

- define a pushdown automaton that accepts the language
- define a grammar for this language



By modifying the grammar from task 2:

$$S \rightarrow A \mid \epsilon$$

$$A \rightarrow aAc \mid B \mid \epsilon$$

$$B \rightarrow bBc \mid \epsilon$$

so that the empty productions are replaced by c , we obtain a grammar that always adds one extra c to the language:

$$S \rightarrow A \mid c$$

$$A \rightarrow aAc \mid B \mid c$$

$$B \rightarrow bBc \mid c$$

then modify this so that the nonterminal C can produce additional c 's:

$$S \rightarrow A \mid C$$

$$A \rightarrow aAc \mid C \mid B$$

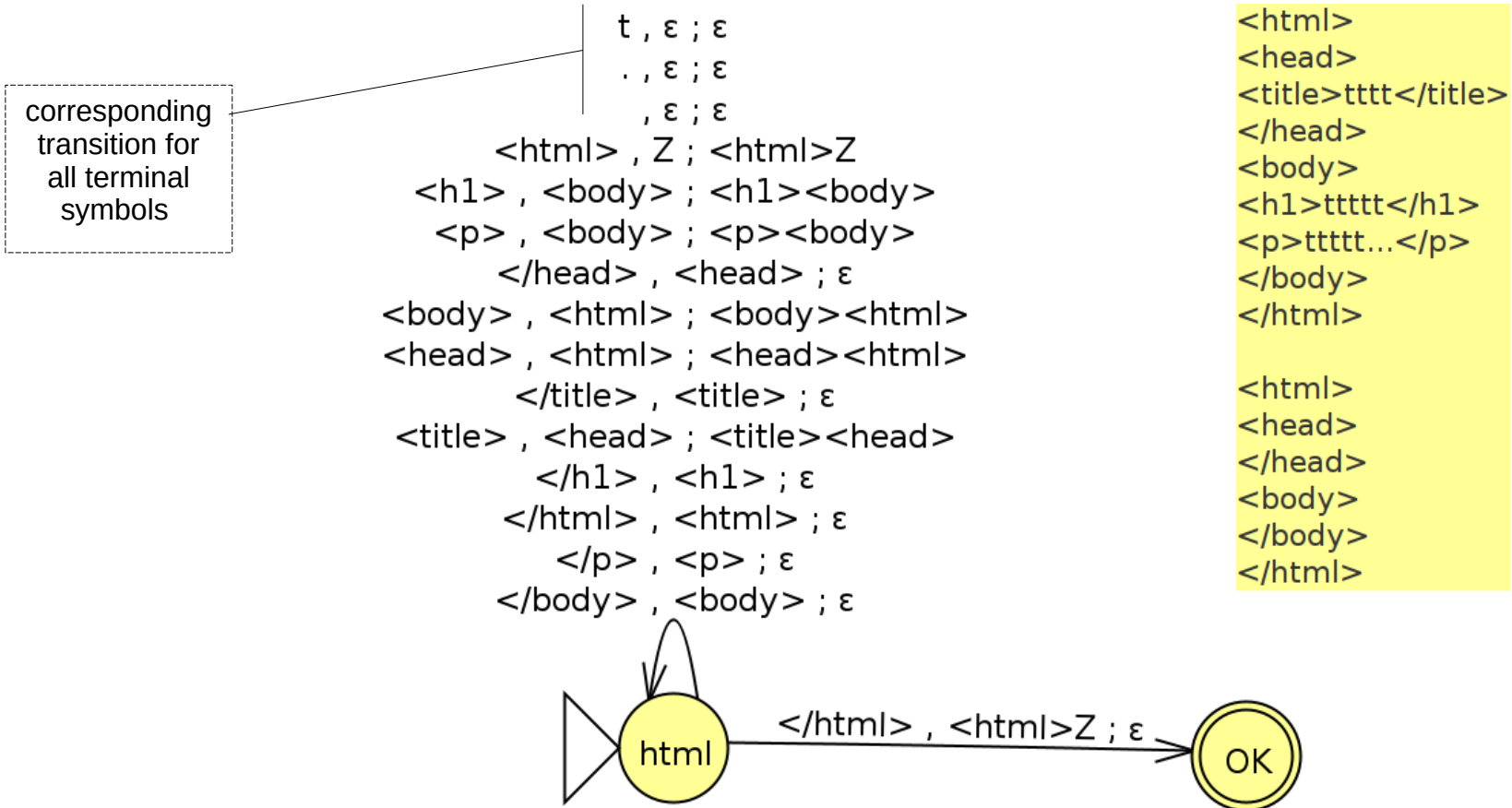
$$B \rightarrow bBc \mid C$$

$$C \rightarrow cC \mid c$$

X4 T4. In elearn, there is a "start" for a pushdown automaton that checks the correctness of HTML syntax. https://www.w3schools.com/html/html_intro.asp

Complete it so that it accepts the following HTML parts

<head>
<title>
<body>
<h1>
<p>



X4 T5. Consider the grammar

$\langle \text{statement} \rangle \rightarrow \langle \text{if-then-else} \rangle \mid \langle \text{if-then} \rangle \mid s$
 $\langle \text{if-then-else} \rangle \rightarrow \text{if } c \text{ then } \langle \text{statement} \rangle \text{ else } \langle \text{statement} \rangle$
 $\langle \text{if-then} \rangle \rightarrow \text{if } c \text{ then } \langle \text{statement} \rangle$

The alphabet Σ of this grammar

$= \{ \text{if, then, else, } c, s \}$.

The alphabet thus also includes keywords composed of multiple characters (if, then, else). For readability, spaces are added between terminal symbols.

a) Derive the string

if c then if c then s else s

with the given grammar, i.e., present one possible derivation chain starting from the nonterminal $\langle \text{statement} \rangle$ and ending with this string.

b) Using the same grammar, present two different parse trees for the above string.

c) Briefly explain why the two trees given in b) differ from each other and why this shows that the grammar is ambiguous

1.

$\langle \text{statement} \rangle$
 $\rightarrow \langle \text{if-then-else} \rangle$
 $\rightarrow \text{if } c \text{ then } \langle \text{statement} \rangle \text{ else } \langle \text{statement} \rangle$
 $\rightarrow \text{if } c \text{ then } \langle \text{if-then} \rangle \text{ else } \langle \text{statement} \rangle$
 $\rightarrow \text{if } c \text{ then if } c \text{ then } \langle \text{statement} \rangle \text{ else } \langle \text{statement} \rangle$
 $\rightarrow \text{if } c \text{ then if } c \text{ then } s \text{ else } s$

2.

$\langle \text{statement} \rangle$
 $\rightarrow \langle \text{if-then} \rangle$
 $\rightarrow \text{if } c \text{ then } \langle \text{statement} \rangle$
 $\rightarrow \text{if } c \text{ then } \langle \text{if-then-else} \rangle$
 $\rightarrow \text{if } c \text{ then if } c \text{ then } \langle \text{statement} \rangle \text{ else } \langle \text{statement} \rangle$
 $\rightarrow \text{if } c \text{ then if } c \text{ then } s \text{ else } s$

X4 T5. Consider the grammar

$\langle \text{statement} \rangle \rightarrow \langle \text{if-then-else} \rangle \mid \langle \text{if-then} \rangle \mid s$

$\langle \text{if-then-else} \rangle \rightarrow \text{if } c \text{ then } \langle \text{statement} \rangle \text{ else } \langle \text{statement} \rangle$

$\langle \text{if-then} \rangle \rightarrow \text{if } c \text{ then } \langle \text{statement} \rangle$

The alphabet Σ of this grammar

$= \{ \text{if, then, else, } c, s \}$.

The alphabet thus also includes keywords composed of multiple characters (if, then, else). For readability, spaces are added between terminal symbols.

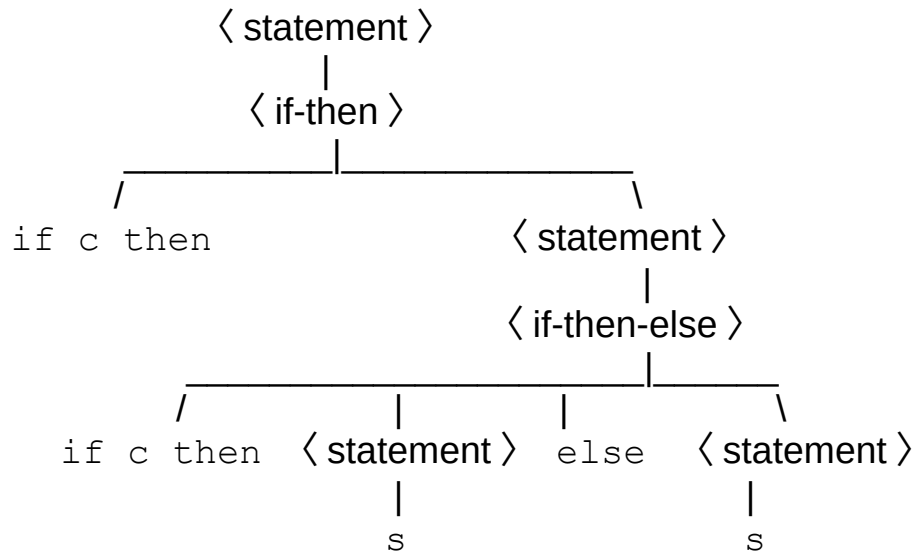
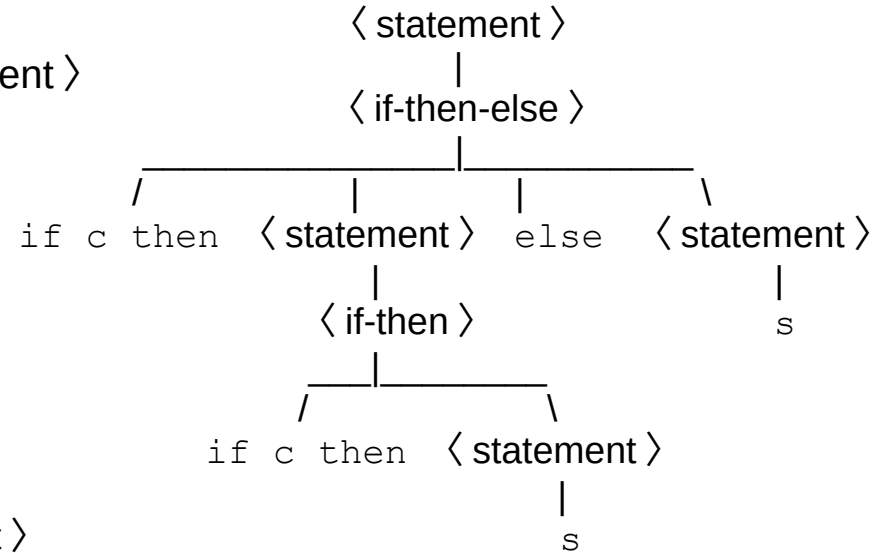
a) Derive the string

if c then if c then s else s

with the given grammar, i.e., present one possible derivation chain starting from the nonterminal $\langle \text{statement} \rangle$ and ending with this string.

b) Using the same grammar, present two different parse trees for the above string.

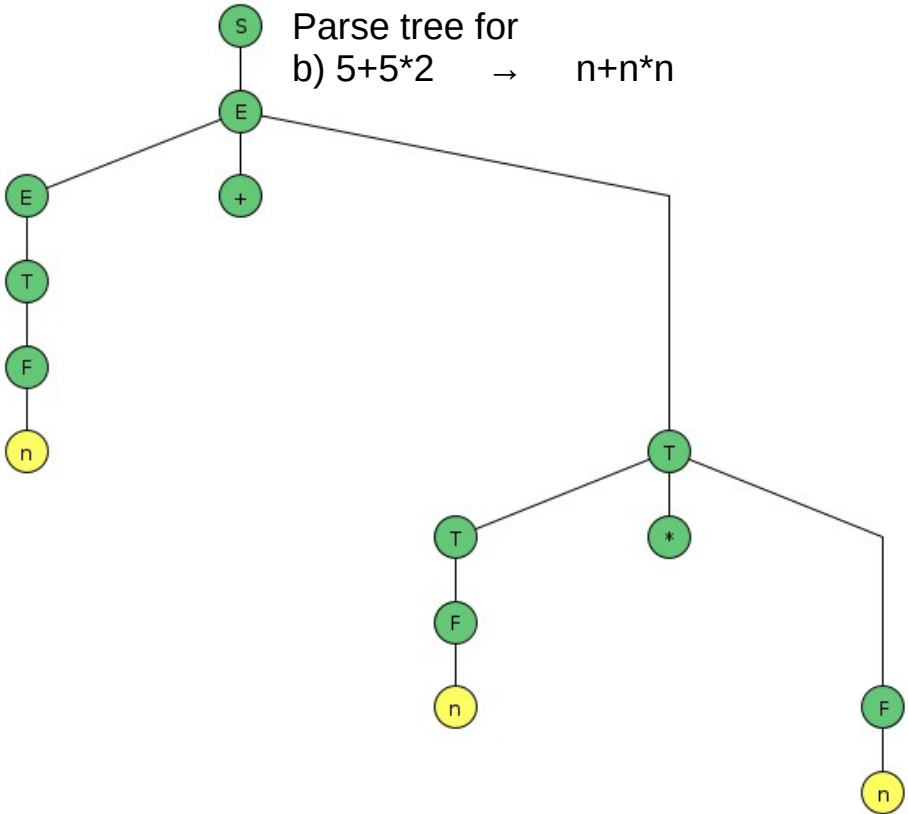
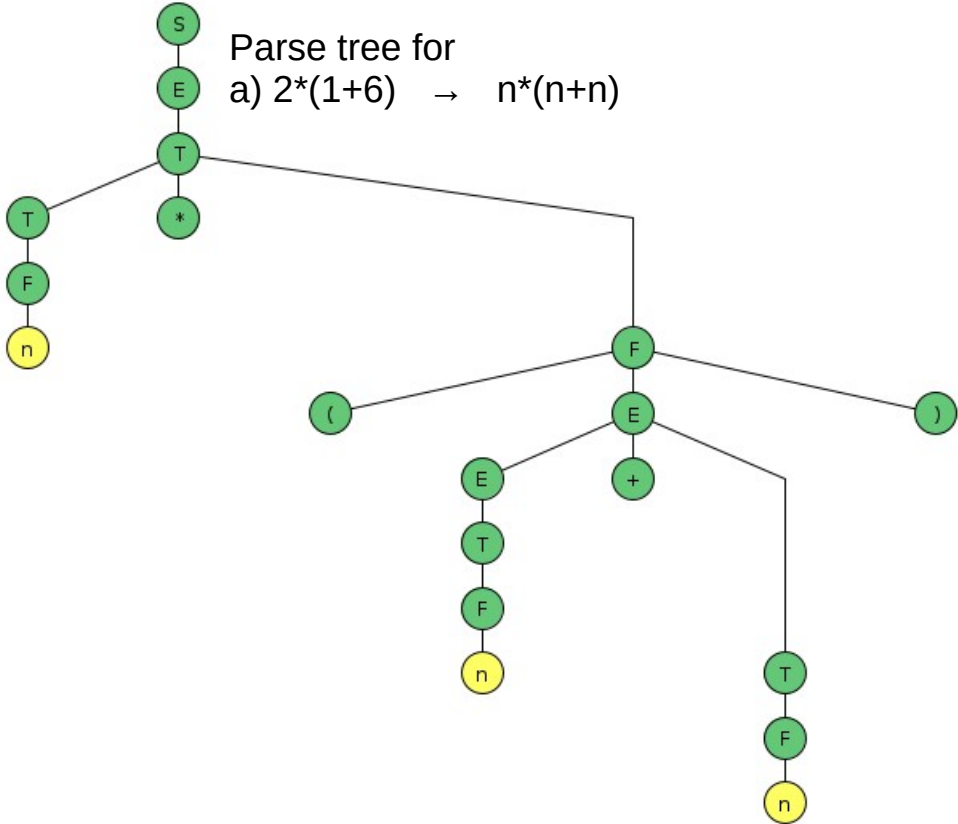
c) Briefly explain why the two trees given in b) differ from each other and why this shows that the grammar is ambiguous



X4 T6. Grammar of arithmetic expressions. An arithmetic expression (E) contains terms (T), factors(F) and numbers (n). Hence, n can be any number. Consider a somewhat simple grammar $\Sigma = \{ +, *, (,), n \}$

$S \rightarrow E$
 $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow n \mid (E)$

In this grammar, the lower a nonterminal is, the stronger it binds. F handles numbers and parentheses (E). Since T is below E in the grammar, * binds more tightly than +, so + is applied last



X4 T6. Grammar of arithmetic expressions. An arithmetic expression (E) contains terms (T), factors(F) and numbers (n). Hence, n can be any number. Consider a somewhat simple grammar $\Sigma = \{ +, *, (,), n \}$

$S \rightarrow E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow n \mid (E)$

Present parse trees for:

a) $n^*(n+n)$

b) $n+n*n$

c) $n*n+n$

